



MINISTÉRIO DA EDUCAÇÃO
Universidade Federal de Alfenas . UNIFAL-MG
Av. Jovino Fernandes de Sales, 2600 - Santa Clara, Alfenas - MG, 37130-000
Ciência da Computação



Universidade Federal de Alfenas

Vinicius Ribeiro da Silva do Carmo

Análise da Implementação do Protocolo Go-Back-N em Java via UDP

Alfenas, Minas Gerais

2026



Introdução.....	3
Tecnologias Utilizadas.....	3
Decisões de Projeto e Arquitetura.....	4
Concorrência e Gestão do Temporizador (Timeout).....	4
Estabelecimento de Sessão e a Dúvida do Handshake.....	5
Estruturação do Datagrama e a escolha do ByteBuffer.....	5
Gravação de Arquivos e o problema do "Byte Lixo".....	6
Validação de Integridade: MD5 versus SHA-256.....	6
Dificuldades Encontradas e Aprendizados.....	6
O Desafio do Assincronismo: A Armadilha do "Stop-and-Wait".....	7
Implementação da simulação de perda de pacotes.....	7
Conclusão e Resultados.....	7
Cenário A: Impacto do Paralelismo (Tamanho da Janela).....	8
Resultados obtido:.....	8
Cenário B: O Efeito da Perda de Pacotes e a Eficiência do GBN.....	9
Resultados obtido:.....	9
Cenário C: Teste de Estresse e Robustez do Sistema.....	10
Resultados obtido:.....	10
Conclusão.....	10
Referências Bibliográficas.....	11



Introdução

A camada de transporte da arquitetura da Internet oferece dois modelos principais de serviço: o TCP, que provê uma comunicação confiável orientada à conexão, e o UDP, um serviço de melhor esforço (*best-effort*) que prioriza a agilidade, mas não garante a entrega, a ordenação ou o controle de congestionamento dos datagramas. Este trabalho tem como objetivo consolidar os conceitos teóricos de transferência confiável de dados através da construção de um sistema de transferência de arquivos construído integralmente sobre o protocolo UDP.

Para contornar a natureza inerentemente não confiável do meio físico e do UDP, este projeto implementa, na camada de aplicação, os mecanismos lógicos do protocolo de janela deslizante Go-Back-N (GBN). A arquitetura do sistema foi dividida em dois componentes principais independentes:

- Um Emissor (Cliente): Responsável por ler um arquivo local, particioná-lo, despachar múltiplos datagramas simultâneos (limitados pelo tamanho da janela) e gerenciar retransmissões automáticas de pacotes não confirmados com base em um temporizador de *timeout*.
- Um Receptor (Servidor): Encarregado de aceitar a conexão, validar o sequenciamento dos pacotes recebidos, emitir confirmações cumulativas (ACKs) e gravar o arquivo final. O receptor atua também como um simulador de falhas de rede, descartando pacotes artificialmente com base em uma probabilidade parametrizável para validar a robustez do mecanismo de retransmissão.

Por fim, a exatidão da transferência é comprovada por um sistema de validação de integridade que compara a assinatura criptográfica final do arquivo enviado com o arquivo recebido.

Tecnologias Utilizadas

O sistema foi desenvolvido de forma nativa utilizando a linguagem Java e o ecossistema de construção Maven. O projeto não utiliza *frameworks* de terceiros, apoiando-se exclusivamente nas bibliotecas padrão da linguagem, destacando-se:

- `java.net` (`DatagramSocket` e `DatagramPacket`): Para a comunicação em baixo nível baseada em datagramas UDP.



- `java.nio (ByteBuffer)`: Para a serialização otimizada dos campos do cabeçalho binário customizado da aplicação.
- `java.util.concurrent (Threads e ScheduledExecutorService)`: Para gerenciar o paralelismo da janela deslizante, permitindo o envio de dados, a escuta de ACKs e a contagem de tempo de forma concorrente e sem bloqueios.
- `java.security (MessageDigest)`: Para o cálculo e validação final dos hashes MD5 dos arquivos.

Decisões de Projeto e Arquitetura

O desenvolvimento de um mecanismo de transferência de dados confiável sobre UDP exigiu a construção manual de lógicas que, normalmente, são abstraídas pelo sistema operacional durante o uso do TCP. Durante a fase de planejamento, surgiram diversas dúvidas metodológicas e desafios arquiteturais. Abaixo, detalho como cada ponto foi abordado e as justificativas para as escolhas tecnológicas finais.

Concorrência e Gestão do Temporizador (*Timeout*)

A implementação original de uma Janela Deslizante (GBN) não pode ser bloqueante. A primeira abordagem considerada foi utilizar o método `setSoTimeout()` do socket nativo para gerenciar as perdas de pacotes. Contudo, após algumas pesquisas, concluí que isso transformaria a arquitetura em um protocolo de pare-e-espere (*Stop-and-Wait*), destruindo o propósito do paralelismo e a alta vazão do GBN.

- **Decisão:** optei por separar a arquitetura do Emissor em três frentes de execução concorrentes: uma *Thread* principal (para leitura do arquivo e envio de dados), uma *Thread* secundária (dedicada exclusivamente à escuta contínua de ACKs) e um *ScheduledExecutorService* (para disparar e gerenciar o timeout do pacote base).
- **Otimização de CPU:** Para evitar o problema de *busy-waiting* (em que a *Thread* principal consumiria 100% do processamento ao verificar repetidamente em um loop se há espaço na janela), implementei o uso de



blocos sincronizados com `wait()` e `notifyAll()`. A *Thread* de envio "adormece" assim que a janela atinge sua capacidade máxima e só é acordada quando a *Thread* de recepção processa um novo ACK válido.

Estabelecimento de Sessão e a Dúvida do Handshake

No início do desenvolvimento, surgiu um questionamento crítico em relação ao Handshake inicial: "se o Emissor enviar o pacote de Handshake e começar imediatamente a despejar os pacotes de Dados na rede, o que acontece se o Handshake for perdido?"

Havia duas rotas possíveis: ignorar a perda (assumindo a falha da transferência) ou implementar um reconhecimento específico pré-transferência.

- **Decisão:** após analisar os dois cenários, optei por implementar um "ACK de Handshake" exclusivo. O Emissor entra em um estado de bloqueio logo após o envio dos parâmetros iniciais (tamanho do arquivo, caminho e probabilidade de perda) e aciona um temporizador. Ele só avança para a fase de envio de dados após o Receptor confirmar que interceptou o Handshake, criou o arquivo de destino e está pronto para receber a carga útil. Isso evitou o envio massivo de pacotes de dados para o vácuo.

Estruturação do Datagrama e a escolha do ByteBuffer

Como os *Sockets* UDP transmitem apenas *arrays* de *bytes* puros, existia o desafio de fatiar valores inteiros de 32 *bits* (como o Número de Sequência e o ACK) e valores curtos de 16 *bits* (como o Tamanho dos Dados) para acomodá-los no cabeçalho customizado de 11 *bytes*.

- **Decisão:** Em vez de utilizar operações binárias manuais de deslocamento de *bits*, que tornariam a leitura do código mais densa e propensa a erros, decidi seguir a recomendação do professor e utilizei a classe `java.nio.ByteBuffer`. Essa estrutura funcionou como uma fôrma inteligente, permitindo a serialização e desserialização direta dos tipos primitivos do Java de forma elegante e blindada contra falhas de tipagem.



Gravação de Arquivos e o problema do "Byte Lixo"

Uma falha recorrente em sistemas de transferência que operam em blocos fixos de tamanho máximo (1024 *bytes*) é o corrompimento do arquivo em sua extremidade final. Isso ocorre porque o último pacote geralmente não preenche toda a capacidade de carga, resultando na gravação de espaços nulos (*bytes* zerados).

- **Decisão:** O campo `tamanhoDados`, implementado no cabeçalho do datagrama, foi utilizado como uma régua para a extração. O Receptor foi programado para ler esse valor dinamicamente em cada iteração e escrever no disco apenas a quantidade exata de *bytes* úteis contidos no *array* recebido, garantindo uma reconstrução milimétrica do arquivo de origem.

Validação de Integridade: MD5 versus SHA-256

Embora eu reconheça que o SHA-256 é amplamente considerado o padrão-ouro de mercado e a escolha ideal para projetos que exigem segurança contra ataques maliciosos, fui atrás de conhecer e compreender a decisão de utilizar o MD5 neste cenário específico, o que gerou uma análise sobre qual algoritmo seria mais adequado para os objetivos desta implementação e o porquê do professor ter recomendado o MD5.

- **Decisão:** foi implementado via API `java.security.MessageDigest`. Como o foco deste trabalho é estritamente a detecção de corrompimento de dados e perdas de transmissão causadas por falhas físicas, o MD5 cumpre o papel com excelente desempenho. Além disso, a assinatura de apenas 32 caracteres do MD5 oferece uma vantagem prática de legibilidade, facilitando a comparação visual nos terminais durante a demonstração do projeto.

Dificuldades Encontradas e Aprendizados

A transição da teoria do modelo Go-Back-N (GBN) visto em sala de aula para uma implementação prática em Java revelou desafios técnicos que exigiram refatorações no planejamento inicial do sistema. Abaixo, destacam-se os principais obstáculos enfrentados e como foram solucionados.



O Desafio do Assincronismo: A Armadilha do "Stop-and-Wait"

A maior dificuldade estrutural do trabalho foi lidar com o assincronismo exigido pela janela deslizante, como já mencionado anteriormente neste relatório. Se a thread de envio ficasse travada esperando um *timeout*, a aplicação seria incapaz de enviar os pacotes subsequentes da janela. Isso rebaixaria o protocolo para um modelo Pare-e-Espere (rdt 3.0), destruindo a eficiência do *pipelining* do GBN. A quebra desse paradigma exigiu o estudo e a implementação de programação concorrente com múltiplas Threads (uma para envio contínuo e outra para escuta de ACKs), adicionando complexidade ao gerenciamento de estados.

Implementação da simulação de perda de pacotes

A implementação da simulação de perda de pacotes no Receptor pareceu trivial à primeira vista (um simples sorteio aleatório via `Math.random()`). Contudo, o desafio surgiu ao decidir a localização exata dessa regra no código. Se o pacote fosse descartado *antes* de avaliarmos o seu Número de Sequência, o Receptor perderia a capacidade de identificar os pacotes "fora de ordem", prejudicando a estatística final. Percebi também que o simulador só poderia aplicar o descarte (e contabilizar uma "perda de rede") caso o pacote recém-chegado fosse exatamente o esperado (`expectedSeqNum`). Pacotes que chegam fora de ordem já representam falhas tratadas pelas regras do próprio GBN (reenviando o ACK anterior) e separá-los da estatística de perdas artificiais foi vital para a exatidão dos resultados.

Conclusão e Resultados

Para validar a implementação da máquina de estados do Go-Back-N e avaliar o seu desempenho em diferentes condições de rede, foram gerados arquivos binários de tamanhos exatos utilizando comandos nativos do sistema operacional (como o `fsutil` no Windows / `dd` em sistemas Unix). Os testes foram segmentados em três cenários de execução, aferindo o impacto do paralelismo, a degradação sob perdas e a estabilidade estrutural do sistema sob estresse.

A validação de integridade em todos os testes operou com sucesso: em todas as execuções, as assinaturas criptográficas (Hashes MD5) impressas pelos terminais do



Emissor e do Receptor corresponderam de forma idêntica, comprovando a confiabilidade assíncrona do protocolo.

Cenário A: Impacto do Paralelismo (Tamanho da Janela)

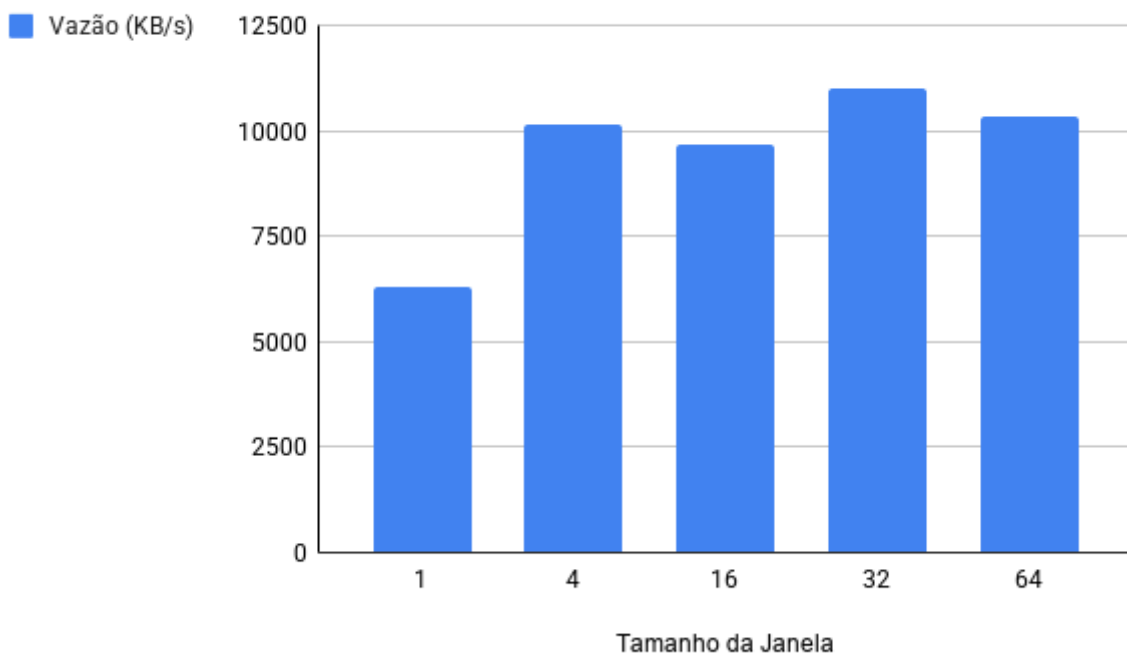
Neste cenário, utilizei um arquivo de 1MB (teste1MB.bin), fragmentado em exatamente 1024 pacotes. A rede foi configurada como ideal (0% de perda simulada), variando-se exclusivamente o tamanho da janela de transmissão nos comandos de execução:

Exemplo do comando de teste base:

```
java -cp target/go-back-n-0.0.1-SNAPSHOT.jar com.example.gobackn.Emissor  
teste1MB.bin 127.0.0.1:recebido.bin [JANELA] 0.0
```

Resultados obtido:

Cenário A - Vazão por Tamanho da Janela



Nesta primeira bateria de testes, os resultados revelaram que a configuração com janela de tamanho 1 impôs o pior desempenho do sistema, atuando de maneira idêntica a um protocolo *Stop-and-Wait*, no qual o emissor é obrigado a cessar o envio e aguardar o tempo de ida e volta (RTT) de cada datagrama individual antes de despachar o próximo bloco.



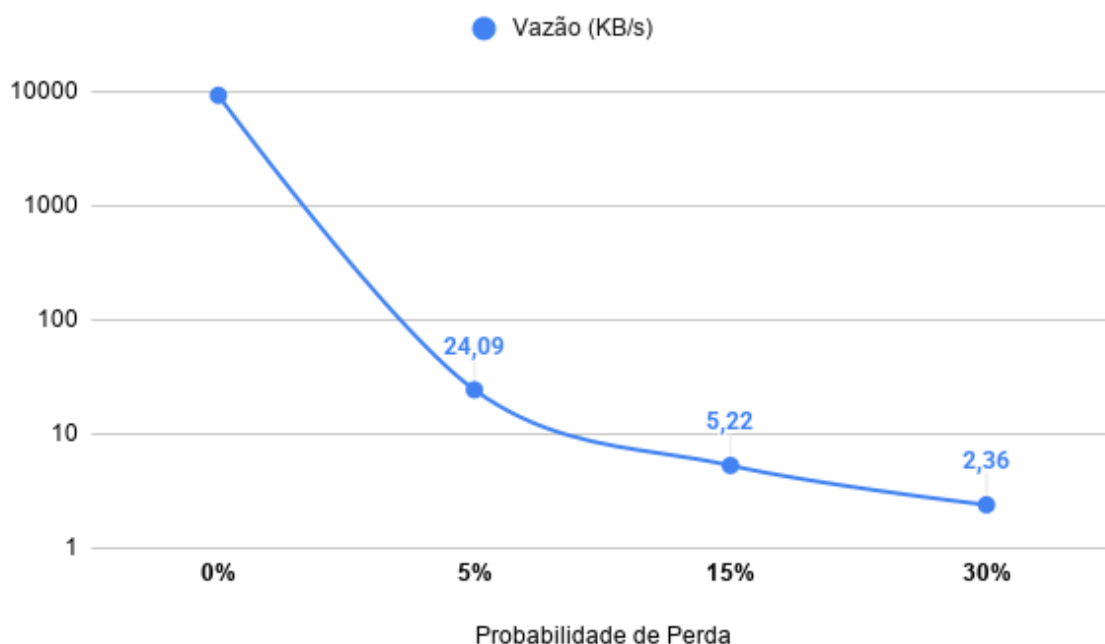
Com o aumento escalonado do tamanho da janela para 4 e 16, por exemplo, observei uma redução drástica no tempo total de execução e uma elevação diretamente proporcional na vazão (*throughput*). Esse comportamento comprova empiricamente os benefícios do *pipelining* (paralelismo) introduzido pela janela deslizante, permitindo que múltiplos pacotes trafeguem e ocupem o canal de comunicação simultaneamente antes do recebimento de suas respectivas confirmações. Ao atingir a configuração de janela igual a 64, o incremento de desempenho demonstrou uma curva de estabilização (platô), indicando que o gargalo de transmissão deixou de ser o tamanho da janela e passou a ser limitado pela capacidade local de processamento de I/O de disco e pelo escalonamento de *threads* do sistema operacional na interface de *loopback*.

Cenário B: O Efeito da Perda de Pacotes e a Eficiência do GBN

Para avaliar o comportamento da aplicação em cenários com ruídos de rede, fixei o tamanho da janela em 16 pacotes sobre o mesmo arquivo de 1MB, injetando taxas progressivas de erro no simulador de perdas do Receptor (0%, 5%, 15% e 30%). Os logs gerados revelaram o impacto severo que o descarte de datagramas impõe sobre o mecanismo de controle de fluxo do Go-Back-N.

Resultados obtido:

Cenário B - Impacto da Probabilidade de Perda na Vazão





Sob uma taxa de perda severa de 30%, o volume de pacotes redundantes injetados na rede cresceu de forma exponencial, e a vazão útil desabou. A explicação teórica para esse fenômeno reside na Máquina de Estados Finitos (FSM) do receptor GBN: por não possuir um buffer para armazenar cargas fora de ordem, o descarte artificial de um único pacote (por exemplo, o pacote de sequência K) invalida toda a janela subsequente de 15 pacotes que já estava em trânsito. O Receptor descarta sistematicamente todos os blocos seguintes como "fora de ordem" e força o reenvio do último ACK válido. Assim que o cronômetro do pacote base estoura no Emissor através do `ScheduledExecutorService`, este é obrigado a "voltar N casas" e retransmitir toda a janela pendente a partir do buffer circular em memória RAM. Esse comportamento demonstra de forma prática o desperdício de largura de banda inerente ao GBN, justificando porque os protocolos modernos de larga escala (como o TCP) adotam lógicas de Reconhecimento Seletivo (*Selective Repeat*).

Cenário C: Teste de Estresse e Robustez do Sistema

O último cenário consistiu na transferência de um arquivo de alta densidade com 10MB (`teste10MB.bin`) fatiado em exatos 10.240 pacotes, sob uma janela de tamanho 8 e perda simulada de 10%. Esse teste teve como objetivo avaliar a estabilidade do código sob regime de alta concorrência e carga massiva de leitura e escrita em disco.

Resultados obtido:

O log de execução demonstrou que o Emissor realizou mais de 9.000 retransmissões para compensar as quedas silenciosas provocadas pelo simulador do Receptor, resultando em um tráfego consolidado que superou a marca de 19 mil datagramas UDP processados. O sistema superou o estresse contínuo sem apresentar interrupções, condições de corrida (*race conditions*), travamentos mútuos (*deadlocks*) ou esgotamento de recursos por vazamento de memória (*memory leaks*). A coordenação entre frentes de execução concorrentes mediada por `synchronized`, `lock.wait()` e `lock.notifyAll()` manteve o Emissor responsivo, liberando dinamicamente os slots da janela circular à medida que os ACKs cumulativos limpavam o `BufferJanela`.

Conclusão

A implementação prática desenvolvida neste trabalho cumpriu com êxito todos os objetivos propostos. A fragilidade nativa do protocolo UDP foi completamente mitigada na camada de aplicação através do controle rigoroso de números de sequência, timeouts precisos e retransmissões automatizadas. A prova do sucesso do protocolo foi a validação final da integridade: em todos os cenários de teste —



mesmo sob condições caóticas de perda de pacotes —, o hash criptográfico MD5 calculado sobre o arquivo gerado no Receptor coincidiu caractere por caractere com a assinatura do arquivo original do Emissor. O projeto permitiu materializar os conceitos abstratos de sistemas de transporte confiáveis, evidenciando as complexidades de sincronização assíncrona na engenharia de protocolos de redes de computadores.

Referências Bibliográficas

KUROSE, James F.; ROSS, Keith W. Redes de Computadores e a Internet: uma abordagem top-down. 8. ed. São Paulo: Pearson Education, 2021.

INSTITUTO DE INFORMÁTICA (UFRGS). *Redes de Computadores: Protocolos de Janela Deslizante (Go-Back-N)*. Disponível em: [UFRGS - Instituto de Informática](#).

FAGUNDES, Eduardo M. *Protocolo usando Go-Back-N*. Disponível em: <https://efagundes.com/networking/protocolos-elementares-de-enlace-de-dados/protocolo-usando-go-back-n/>.

TREINAWEB. *Uma introdução a TCP, UDP e Sockets*. Disponível em: <https://www.treinaweb.com.br/blog/uma-introducao-a-tcp-udp-e-sockets>.

DEVMEDIA. *Java Sockets: Criando comunicações em Java*. Disponível em: <https://www.devmedia.com.br/java-sockets-criando-comunicacoes-em-java/9465>.

RODOLFO, Claudio. *Programação Distribuída - Socket UDP em Java*. YouTube, 7 de out. de 2020. Disponível em: [Programação Distribuída - Socket UDP em Java](#)

ORACLE. *ByteBuffer (Java Platform SE 8)*. Oracle Help Center, 2023. Disponível em: <https://docs.oracle.com/javase/8/docs/api/java/nio/ByteBuffer.html>.

ORACLE. *ScheduledExecutorService (Java Platform SE 8)*. Oracle Help Center, 2023. Disponível em: <https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/ScheduledExecutorService.html>.

ORACLE. *Java Platform, Standard Edition API Specification. Versão 21. 2023*. Disponível em: <https://docs.oracle.com/en/java/javase/21/docs/api/>.